

ПРОГРАМИРАЊЕ 2

- припремни задаци за домаћи задатак 1 -

Саставити програм на програмском језику **C** којим се врши одређена обрада према поставци задатка. Главни програм треба да:

1. позива функције које учитавају све потребне податке са стандардног улаза према описаном формату у тексту задатка;
2. позива одговарајуће функције које изврше захтевану обраду;
3. позива одговарајуће функције које исписује све добијене резултате;

Водити рачуна о **декомпозицији програма на потпрограме** према захтевима задатка и горе описаној расподели. По потреби, дозвољено је уводити и додатне потпрограме у решење. Студент сам треба да дефинише имена функција, као и аргументе потребне за њихов рад. Водити рачуна да се функцијама доставе само неопходни подаци. Потпрограми не смеју приступати променљивама главног програма директно, већ само путем својих аргумената и повратне вредности.

Приликом решавања задатка, користити динамичку алокацију меморије за смештање коришћене структуре података (низа или матрице). Уколико задате димензије структуре података нису коректне, потребно је коректно прекинути програм и не исписивати ништа. Тип елемената структуре података одабрати према потребама задатка, односно користити произвољни тип тамо где то није суштински битно за сам алгоритам. У случају неуспешне доделе динамичке меморије, исписати поруку о грешци (**MEM_GRESKA**) и коректно прекинути извршавање програма. Након завршетка програма (унос, обрада, испис) деалоцирати сву коришћену динамичку меморију.

Програм треба да чита податке уз вођење рачуна о типу података који се чита. Сви програми читају податке са стандардног улаза према формату који је задат текстом конкретног домаћег задатка. Сматрати да су сви подаци на стандардном улазу наведени у исправном формату према тексту задатка, осим уколико другачије није наглашено у тексту конкретног задатка. Сматрати да су коректне димензије низова или матрица целобројне вредности веће од 0. Приликом обраде специјалних или некоректних случајева, обратити пажњу само на оне наведене у поставци задатка.

Код исписа резултата, строго **водити рачуна о формату исписа** који је дефинисан текстом конкретног домаћег задатка. Све реалне бројеве у оквиру исписа заокруживати на две децимале. Програм у свим ситуацијама треба да коректно заврши своје извршавање и врати вредност 0 оперативном систему.

Напомене:

- а)** Домаћи задатак се вежба и предаје путем *Moodle* платформе за електронско учење. Приликом предаје домаћег задатка за сваког студента се на насумичан начин бира једна од варијанти припремних домаћих задатака из овог документа.
- б)** На *Moodle* курсу предмета доступни су тестови за вежбу домаћег задатка.
- в)** Термин предаје домаћег задатка ће бити благовремено објављен.
- г)** Домаћи задатак се решава самостално. Предметни наставници задржавају право да након предаје домаћег задатка изврше проверу сличности и предузму одговарајуће дисциплинске мере. Током израде решења није дозвољена употреба алата вештачке интелигенције заснованих на великим језичким моделима (*ChatGPT*, *Github Copilot* и сл.).

0. Написати програм који врши обрачунавање утрошених телефонских импулса у оквиру мобилне мреже једног малог предузећа задатог у виду низа разговора. Импулс представља обрачунски период изражен у секундама. Запослени у оквиру предузећа се идентификују претплатничким телефонским бројем (цео број) чије последње две цифре јединствено идентификују запосленог. Сматрати да предузеће нема више од 100 запослених. Програм најпре учитава укупан број обављених разговора, а затим и саме податке о разговорима. За сваки разговор се памте информације о претплатничком броју са кога је вршен позив, као и трајање позива у секундама (цео број). Најпре се у једном реду учитавају информације о претплатничким бројевима, а затим у наредном реду трајања одговарајућих позива у секундама. Након тога се, у новом реду, учитава обрачунски период (цео број) у секундама (трајање импулса). Програм треба да за сваког запосленог израчуна број утрошених импулса, као и укупан број утрошених импулса за цело предузеће. Приликом обрачунавања броја импулса, сматрати да је импулс утрошен, уколико је започета бар једна секунда обрачунског периода. На стандардни излаз исписати најпре податке о укупном броју утрошених импулса за цело предузеће, а затим за сваког запосленог који је утрошио барем један импулс у засебном реду две цифре које га идентификују и број утрошених импулса.

Програм реализовати према следећим ставкама:

- 1) Учитати дужину низа разговора. Проверити да ли је дужина низа коректна (> 0) и коректно завршити програм (враћањем вредности 0), уколико није. Учитати елементе низа разговора, најпре информације о претплатничким бројевима, затим информације о трајању позива.
- 2) Проверити да ли су унети претплатнички бројеви у опсегу [0-99], да ли су трајања позива позитивни бројеви и коректно завршити програм (враћањем вредности 0), уколико нешто од претходно наведеног није испуњено.
- 3) Исписати податке о претплатничким бројевима и податке о трајању позива у појединачним редовима, при чему су подаци у једном реду раздвојени једним бланко знаком.
- 4) Обрачунати број утрошених импулса за сваког запосленог, као и укупан број утрошених импулса за цело предузеће, који се враћа као повратна вредност и исписује у новом реду на стандардном излазу.
- 5) Исписати информације о броју утрошених импулса за све запослене са барем једним утрошеним импулсом у појединачним редовима (по растућој вредности претплатничког броја), при чему се у сваком реду исписује претплатнички број запосленог и број утрошених импулса одвојених једним бланко знаком.

Примери

Стандардни улаз	Стандардни излаз
7 23 13 15 13 15 23 23 3 5 7 9 11 12 16 3	23 13 15 13 15 23 23 3 5 7 9 11 12 16 23 13 5 15 7 23 11
10 0 3 4 1 1 9 0 1 5 5 43 25 25 17 44 14 31 23 28 18 5	0 3 4 1 1 9 0 1 5 5 43 25 25 17 44 14 31 23 28 18 57 0 16 1 18 3 5 4 5 5 10 9 3
-5	

1. Написати програм који распоређује долазеће купце на доступне касе. Купци су представљени бројем артикала који носе са собом и распоређују се у редоследу којим долазе и то на прву слободну касу, редом којим су наведене. Сматрати да је за обраду сваког артикла потребно исто време, као и да свака каса обрађује појединачне артикле за исто време. У почетном тренутку су све касе слободне.

Програм реализовати према следећим ставкама:

- 1) Учитати број купаца. Проверити да ли је дужина низа коректна (> 0) и коректно завршити програм (враћањем вредности 0), уколико није.
- 2) Учитати информације о купцима. Проверити за сваког купца да ли је број артикала позитивна вредност и коректно завршити програм (враћањем вредности 0), уколико то није случај.
- 3) Учитати број каса. Проверити да ли је унета вредност коректна (> 0) и коректно завршити програм (враћањем вредности 0), уколико није.
- 4) Извршити распоређивање купаца према критеријуму наведеном у поставци задатка и исписати распоред купаца по касама. Свака каса се исписује у засебном реду исписом њених купаца који су раздвојени тачно једним бланко знаком.

Примери

Стандардни улаз	Стандардни излаз
6 5 4 8 9 2 4 3	5 2 4 4 9 8
7 5 7 2 4 6 4 3 2	5 2 4 4 7 6 3
-5	

2. Написати програм који проверава да ли је решење Судoku загонетке валидно. Судoku загонетка се игра на табли $N^2 * N^2$, која је подељена на N^2 региона димензија $N * N$ у којој се налазе бројеви у опсегу $[1, N^2]$. Решење Судoku загонетке мора да испуни следеће услове: 1) сва поља садрже тачно један број из одговарајућег опсега, 2) свака колона, ред и регион садржи јединствене бројеве. Програм треба да учита потребне податке, провери да ли је решење загонетке валидно и испише одговарајућу поруку на стандардни излаз. У случају да загонетка не садржи бројеве из одговарајућег опсега програм исписује **LOS OPSEG** и престаје са радом. У случају да загонетка не испуњава јединственост колона, редова или региона потребно је исписати поруку **LOSE KOLONE**, **LOSI REDOVI** или **LOSI REGIONI**, редом. Уколико је решење загонетке добро, програм исписује поруку **VALIDNO RESENJE**.

- 1) Учита број N , на основу кога се одређује величина загонетке. Провери да ли је уčitана вредност коректна (> 0) и прекине извршавање програма, уколико није.
- 2) Позове функцију која учита матрицу са подацима о потенцијалном решењу загонетке.
- 3) Позове функцију која провери да ли су сви елементи уčitане матрице вредности из опсега $[1, N^2]$ и прекине извршавање програма, уколико нису, уз испис одговарајуће поруке.
- 4) Позове функцију која испише уčitану матрицу.
- 5) Позове функцију која провери решење загонетке и испише одговарајуће поруке на стандардни излаз.

Стандардни улаз	Стандардни излаз
3 9 8 5 6 4 7 2 1 3 2 7 6 3 9 1 5 8 4 3 1 4 5 8 2 6 9 7 8 9 3 1 2 4 7 5 6 1 4 7 9 6 5 8 3 2 5 6 2 7 3 8 1 4 9 7 2 9 8 5 3 4 6 1 6 5 1 4 7 9 3 2 8 4 3 8 2 1 6 9 7 5	9 8 5 6 4 7 2 1 3 2 7 6 3 9 1 5 8 4 3 1 4 5 8 2 6 9 7 8 9 3 1 2 4 7 5 6 1 4 7 9 6 5 8 3 2 5 6 2 7 3 8 1 4 9 7 2 9 8 5 3 4 6 1 6 5 1 4 7 9 3 2 8 4 3 8 2 1 6 9 7 5 VALIDNO RESENJE
2 1 1 2 3 2 3 4 4 3 4 1 2 4 2 3 1	1 1 2 3 2 3 4 4 3 4 1 2 4 2 3 1 LOSI REDOVI LOSI REGIONI
4 0 2 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 5 0 9 0 0 0 0 0 0 0 0 0 0 0 0 4 0 0 0 0 0 0 0 3 0 19 0	LOS OPSEG

3. Написати програм који симулира „Игру живота“. На почетку игре корисник задаје матрицу живих и мртвих ћелија. Најпре се читају димензије матрице, а потом и саме ћелије. Уколико је ћелија означена знаком X, она је мртва. Уколико је ћелија означена знаком O, она је жива. Након тога, корисник уноси жељени број итерације игре живота. У свакој итерацији се креира нова матрица живих и мртвих ћелија на основу матрице из претходне итерације и следећих правила:

- Свака жива ћелија са 2 или 3 жива суседа остаје жива у наредној итерацији;
- Свака мртва ћелија са 3 жива суседа постаје жива у наредној итерацији;
- Све остале ћелије постају или остају мртве у наредној итерацији;

Програм најпре треба да испише иницијално стање матрице, а потом да испише стање матрице након сваке итерације.

Програм реализовати према следећим ставкама:

- 1) Учита димензије матрице. У случају да димензије нису коректне (> 0) коректно завршити програм (враћањем вредности 0).
- 2) Позове функцију која ће прочитати иницијалну матрицу ћелија.
- 3) Позове функцију која ће исписати иницијалну матрицу ћелија.
- 4) Учита број итерација. У случају да број итерација није коректан (> 0) коректно завршити програм (враћањем вредности 0).
- 5) За сваку итерација позове функцију које ће израчунати матрицу ћелија за текућу итерација и функцију која ће исписати исту.

Примери

Стандардни улаз	Стандардни излаз
4 3 OOX XOX OOO OOO 2	INITIAL OOX XOX OOO OOO ITERATION0 OOX XXX XXX OXO ITERATION1 XXX XXX XXX XXX
6 6 OXOOOO OOXOOO XXXOOO XOOOOO XXXXOX OXOOOX 3	INITIAL OXOOOO OOXOOO XXXOOO XOOOOO XXXXOX OXOOOX ITERATION0 OXOXXO OOXXXX OXXXXX XXOXXX XXXXXX XXXOOX ITERATION1 OXXXXX OXXXXX OXXXXX XXXXXX XXOXXX XXXXXX ITERATION2 XXXXXX OOXXXX XXXXXX XXXXXX XXXXXX XXXXXX
-1 0	